

Portfolio Optimization Pipeline - Implementation Report

This report documents the updated code package created from scripts-3.zip, FINAL-copy.zip, Risk-Signal Diversification.docx, and the provided Plotly HTML style reference.

Important execution note: the raw ETF datasets required by the original backtest were not included in the chat sandbox, so the full historical strategy run was not executed here. The delivered repository was syntax-checked and the Plotly report generator was smoke-tested on synthetic data.

1. What was implemented

The package keeps the original scripts structure and adds a console-first research layer. The old notebook logic is no longer locked inside FINAL_results.ipynb: it is represented by scripts/runs/final_results_simple.py, the overlay scripts, and the automatic report generator.

The implementation also preserves FINAL-copy additions: the posterior-predictive scenario fan, forecast_risk.csv output, dynamic parameter history, model-CVaR overlay, realized-volatility overlay, and helper code for eligibility-matrix universes.

Area	Files / modules	Purpose
Smart signals	scripts/risk_controls/smart_signals.py	Kelly, risk-reward, GARCH-style volatility clustering, ER, ATR regime, liquidity, breakout, spike, pullback, net move, composite quality.
Train/test control	scripts/backtesting/*, scripts/results/stats.py	Nested validation/test selection plus post-run train/test metrics, rolling Sharpe and rolling Sortino.
Dashboard	scripts/results/interactive_report.py	One-file Plotly HTML dashboard for monthly rebalance diagnostics.
Run integration	scripts/runs/run.py, strategy_run.py, common_postprocess.py	Smart-signal sizing, audit saving, forecast risk saving, dashboard generation.
Notebook replacement	scripts/runs/final_results_simple.py	Runs configs, overlays and dashboards from console.
Configs	advanced_smart_bayesian.yaml, strategy_smart.yaml	Ready-to-run examples for optimizer and strategy workflows.

2. Smart risk-signal layer

The new layer extends the original risk-signal diversification idea. Instead of using only a backward realized-volatility timer and a forward posterior-predictive CVaR timer, the run can now compute a richer quantitative signal stack at every monthly rebalance.

2.1 Asset-level signals

Efficiency ratio (ER): absolute net price movement divided by the sum of absolute path movements over a window. ER close to 1 means a clean directional move; ER close to 0 means noisy/choppy movement.

ATR volatility regime: current ATR proxy divided by long-term ATR proxy. Values above 1 indicate elevated current volatility relative to the long-term baseline.

Liquidity score: recent dollar-volume/market-cap proxy relative to the cross-sectional median. It rewards assets with better tradability.

Breakout strength: current price location inside the recent high-low channel. It identifies whether an asset is near a breakout, mid-channel, or weak.

Normalized spike: latest return divided by rolling volatility. Large absolute spikes are penalized to reduce entry after unstable jumps.

Pullback ratio: distance from recent high divided by recent range. The implementation rewards moderate pullback and avoids extreme collapse.

Net move: rolling cumulative return over the recent window.

2.2 Forward sizing signals

Kelly criterion: for each ETF, fractional Kelly is approximated as mean scenario return divided by scenario variance, clipped to a long-only cap. Negative Kelly becomes zero.

Risk-reward ratio: expected positive scenario payoff divided by expected negative scenario loss. This is computed from the posterior-predictive scenario fan when available; otherwise recent historical returns are used.

Volatility clustering: a dependency-light GARCH(1,1)-style forecast updates conditional variance recursively. It is not a full maximum-likelihood arch package fit, but it captures volatility persistence in a robust and fast way for monthly allocation.

2.3 Portfolio overlay

The portfolio overlay is multiplicative: exposure falls when any major signal becomes weak. The score combines Kelly, risk-reward, volatility forecast and composite signal quality. If `allow_cash_overlay` is true, the remaining exposure is implicitly held in cash/T-bills by letting total target weights sum below 1.

```
overlay_fraction = vol_scale * rr_scale * kelly_scale * quality_scale
weights_adjusted = optimizer_weights * quality * kelly_multiplier * rr_multiplier
if allow_cash_overlay: weights_adjusted *= overlay_fraction
```

3. Train/test and parameter-stability controls

The original concern was that best backtest parameters are often not the best future parameters. The nested engine now splits each already-known historical window into train, validation and internal test sections. Candidates are generated on train, ranked on validation, re-tested on the internal test period, and only then executed in the next out-of-sample month.

The dashboard and stats.py also report overall/train/test metrics after a run, so parameter stability can be reviewed separately from the full-period headline results.

```
python -m scripts.runs.run -c advanced_smart_bayesian.yaml
# outputs: selection_audit.csv, smart_rebalance_audit.csv, train_test_metrics.csv,
interactive_report.html
```

Added metrics include win rate, average drawdown, expectancy, profit factor, average win/loss, rolling Sharpe ratio and rolling Sortino ratio. Existing metrics such as annualized return, volatility, Sharpe, Sortino, Calmar, max drawdown, VaR and CVaR remain available.

4. Plotly HTML dashboard

Every main run now attempts to build `results//interactive_report.html`. The dashboard follows the spirit of the supplied dark Plotly example but is specific to this ETF monthly-rebalancing framework.

Included panels: strategy vs SPY/equal-weight benchmark growth, drawdown, rolling Sharpe/Sortino, return histograms, turnover by rebalance date, eligible-universe heatmap, selected non-zero ETF heatmap, latest selected ETF covariance/correlation matrix with a dropdown, dynamic optimizer parameters, forward model risk signals, smart signal audit and monthly filterable ETF tables.

The monthly tables are sorted by weight and include a minimum-weight filter. This directly addresses the request to inspect non-zero ETF weights and potential monthly gains/returns at each rebalance.

5. How to run

Optimizer version

```
cd portfolio_optimization_pipeline
python -m scripts.runs.run -c advanced_smart_bayesian.yaml
```

Strategy version

```
cd portfolio_optimization_pipeline
python -m scripts.runs.strategy_run -c strategy_smart.yaml
```

Notebook-equivalent version

```
cd portfolio_optimization_pipeline
python -m scripts.runs.final_results_simple --configs main_dyn_off.yaml main_dyn_strong.yaml
```

Rebuild dashboard only

```
python -m scripts.results.interactive_report --result-folder results/advanced_smart_bayesian
```

6. Files added or materially modified

Added: scripts/risk_controls/smart_signals.py, scripts/results/interactive_report.py, scripts/runs/common_postprocess.py, scripts/runs/final_results_simple.py, scripts/configs/advanced_smart_bayesian.yaml, scripts/configs/strategy_smart.yaml, README_UPDATED.md and IMPLEMENTATION_NOTES.md.

Modified: scripts/runs/run.py, scripts/runs/strategy_run.py, scripts/runs/combined_run.py, scripts/runs/combined_strategy_run.py, scripts/runs/new_run.py, scripts/runs/new_run2.py, scripts/results/stats.py and scripts/backtesting/engine_integration.py.

Preserved from FINAL-copy: cvar_target_overlay.py, fast_voltarget.py, report_style.py and regenerate.py, with copies also placed under scripts/overlays or scripts/reporting for structured access.

7. Limitations and next validation step

The code compiles successfully, and the dashboard generator was smoke-tested on synthetic data. The full monthly ETF backtest should be run on your local machine or server where datasets/, results/business_dates.xlsx, NewClosePrice.csv, Volume.csv and the filtered universe files exist.

After the first real run, review interactive_report.html, selection_audit.csv and smart_rebalance_audit.csv. In particular, verify whether the smart overlay improves test-period drawdown and downside metrics without over-reducing CAGR.